

Testes — Diagnóstico Médico API

Visão Geral

Item	Detalhe
Framework	pytest 9.1.1
Plugins	pytest-cov 7.1.0, pytest-asyncio
Total de testes	136
Modo assíncrono	asyncio_mode = auto (pyproject.toml)
Path de descoberta	tests/
Pythonpath	src/ (imports absolutos tipo from app.xxx)
Linters	Ruff (check + format), executado antes dos testes no CI

Organização em **3 camadas**: unitários → API → integração.

```
tests/
├── conftest.py                # Fixtures compartilhadas (14)
├── unit/                      # 13 arquivos, ~105 testes
│   ├── test_config.py
│   ├── test_core_exceptions.py
│   ├── test_domain_enums.py
│   ├── test_domain_exceptions.py
│   ├── test_domain_models.py
│   ├── test_logger.py
│   ├── test_model_registry.py
│   ├── test_predictor.py
│   ├── test_llm_explainer.py
│   ├── test_llm_factory.py
│   ├── test_llm_providers.py
│   ├── test_groq_provider.py
│   └── test_timing_middleware.py
├── api/                       # 3 arquivos, ~25 testes
│   ├── test_predict_api.py
│   ├── test_explain_api.py
│   └── test_analysis_api.py
└── integration/              # 1 arquivo, 3 testes
    └── test_full_pipeline.py
```

Configuração (pyproject.toml)

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
asyncio_mode = "auto"
```

A engine asyncio opera em `Mode.AUTO`: detecta automaticamente testes com `@pytest.mark.asyncio` ou fixtures `async def` e os executa no event loop.

Fixtures Compartilhadas (conftest.py)

Todas as 14 fixtures ficam em `tests/conftest.py` e são automaticamente descobertas por qualquer arquivo de teste no projeto.

Autouse (aplica-se a todos os testes)

Fixture	O que mocka
<code>mock_aws_and_posthog</code>	<code>get_secret_or_env</code> , <code>get_posthog_client</code> , <code>capture_event</code> , <code>capture_request</code> , <code>capture_prediction</code> , <code>capture_llm_request</code>

Nenhuma requisição real a AWS Secrets Manager ou PostHog é feita durante os testes.

Aplicação e Cliente HTTP

Fixture	Descrição
<code>app</code>	Cria a aplicação FastAPI via <code>create_app()</code>
<code>client(app)</code>	Envolve a app num <code>TestClient</code> do Starlette (httpx síncrono)
<code>anyio_backend</code>	Retorna <code>"asyncio"</code> — necessário para <code>anyio</code> em testes

Dados Sintéticos

Fixture	Descrição
<code>dummy_model_path(tmp_path)</code>	Cria diretório <code>models/</code> em temp
<code>dummy_pipeline</code>	Pipeline <code>StandardScaler + LogisticRegression</code> treinado com dados aleatórios
<code>save_dummy_pipeline</code>	Factory — salva o pipeline acima em <code>.pkl</code>
<code>dummy_features_top20</code>	Arquivo JSON com 3 nomes de features
<code>dummy_logistic_model</code>	<code>LogisticRegression</code> treinado com seed 42 (5 features, 100 amostras)

Mocks de Serviço

Fixture	Descrição
<code>mock_predictor</code>	<code>MagicMock.predict()</code> → <code>PCOSPrediction(diagnosis=1, prob=0.85, confidence="Alta", ...)</code>
<code>mock_explainer</code>	<code>AsyncMock.explain()</code> → <code>(Explanation(...), 150)</code>
<code>mock_llm_client</code>	<code>AsyncMock.chat()</code> → <code>LLMResponse(text=..., tokens_used=50)</code>
<code>override_deps</code>	Sobrescreve 4 dependências FastAPI (<code>get_predictor</code> , <code>get_explainer</code> , <code>get_model_registry</code> , <code>get_llm_provider</code>) com mocks. Limpa ao final.

Testes de Unidade

`test_config.py` (4 testes)

Testa a classe `Settings` do Pydantic:

Teste	O que valida
<code>test_default_values</code>	Valores padrão: <code>model_path="models/pcos_model.joblib"</code> <code>, log_level="INFO",</code> <code>llm_provider="openai"</code>
<code>test_custom_values</code>	Variáveis de ambiente via <code>monkeypatch</code> sobrescrevem defaults
<code>test_model_path_is_string</code>	<code>model_path</code> é do tipo <code>str</code>
<code>test_openai_api_key_empty_by_default</code>	Chave vazia por segurança

`test_core_exceptions.py` (7 testes)

Duas classes:

`TestAppException` — `AppException` é a exceção base da aplicação:

- Armazena `message` e `status_code`
- `status_code` default é 400
- Herda de `Exception`

`TestAppExceptionHandler` — Handler registrado no FastAPI:

- Retorna `JSONResponse` com `{"error": message}` e status code correto
- Testado com status 400, 422 e 500

`test_domain_exceptions.py` (6 testes)

Valida que as 3 exceções de domínio podem ser levantadas e capturadas:

- `ModelNotLoadedError`
- `InvalidFeaturesError`
- `LLMRequestError`

`test_domain_models.py` (8 testes)

`TestPCOSPrediction` — Dataclass de saída da predição:

- Criação com diagnóstico positivo (1) e negativo (0)
- Tipos: `diagnosis` é `int`, `probability` é `float`
- Probabilidades extremas (0.0 e 1.0) funcionam

`TestExplanation` — Dataclass de explicação clínica:

- Criação com texto, fatores de risco e insights
- Listas vazias são aceitas
- Múltiplos insights funcionam

`test_domain_enums.py` (7 testes)

`TestDiagnosisStatus` — `NEGATIVE = 0`, `POSITIVE = 1`, nomes corretos.

`TestModelType` — `LOGISTIC_REGRESSION`, `RANDOM_FOREST`, `XGBOOST` com valores e nomes corretos. Enum tem exatamente 3 membros.

`test_logger.py` (3 testes)

`setup_logging()` não levanta exceção em 3 cenários:

- `log_level = "INFO"`
- `log_level = "DEBUG"`
- `posthog_enabled = True` com chave fictícia

`test_model_registry.py` (4 testes)

Usa fixture local `dummy_model` que salva um `LogisticRegression` treinado via `joblib`:

Teste	O que valida
<code>test_load_artifacts_returns_model_when_found</code>	Retorna instância de <code>LogisticRegression</code>
<code>test_load_artifacts_model_has_predict_proba</code>	<code>predict_proba</code> retorna shape <code>(1, 2)</code>
<code>test_load_artifacts_returns_none_when_not_found</code>	Arquivo inexistente → <code>None</code>
<code>test_load_artifacts_caches</code>	Mesmo objeto em chamadas subsequentes

Teste	O que valida
<code>result</code>	(verificação de identidade)

test_predictor.py (11 testes)

Três classes de teste:

• `TestPredictorValidation` — Validação de entrada no service layer:

- Feature negativa → `InvalidFeaturesError` com "negativo" na mensagem
- Todas features válidas → retorna `PCOSPrediction` com sucesso

• `TestConfidenceLabel` — Função auxiliar `_confidence_label`:

- `>= 0.80` → "Alta"
- `[0.60, 0.80)` → "Média"
- `< 0.60` → "Baixa"

• `TestPredictorService` — Predição com modelo real treinado (usa fixture `trained_model` que cria um `LogisticRegression` com as 20 colunas do `FEATURE_COLUMN_MAP`):

Teste	O que valida
<code>test_predict_returns_pcos_prediction</code>	Retorna <code>PCOSPrediction</code> com diagnosis 0/1 e probability em [0,1]
<code>test_predict_includes_confidence</code>	confidence é "Alta", "Média" ou "Baixa"
<code>test_predict_includes_top_features</code>	<code>top_contributing_features</code> tem exatamente 5 itens
<code>test_predict_top_features_have_direction</code>	Cada feature tem direction "positiva" ou "negativa"
<code>test_predict_raises_when_model_not_loaded</code>	Arquivo inexistente → <code>ModelNotLoadedError</code>
<code>test_predict_uses_correct_model_version</code>	<code>model_version == "2.0.0"</code>

test_llm_explainer.py (12 testes, 9 com @pytest.mark.asyncio)

Testa o `LLMExplainerService` que orquestra o prompt → LLM → parse da resposta:

Teste	O que valida
<code>test_explain_parsing_json_response</code>	JSON válido → extrai <code>explanation</code> , <code>risk_factors</code> , <code>insights</code> , <code>tokens</code>
<code>test_explain_strips_markdown_fences</code>	Remove <code>json ...</code>
<code>test_explain_raises_on_provider_error</code>	Provider com erro → <code>LLMRequestError</code>
<code>test_explain_raises_on_bad_json</code>	Resposta não-JSON → <code>LLMRequestError</code>
<code>test_build_prompt_contains_features_and_diagnosis</code>	Prompt inclui <code>features</code> , "POSITIVO", <code>percentual</code>
<code>test_explain_empty_risk_factors_and_insights</code>	Listas vazias são tratadas
<code>test_explain_tokens_used_none</code>	<code>tokens_used = None</code> não quebra
<code>test_explain_raises_on_missing_explanation_key</code>	JSON sem "explanation" → erro
<code>test_explain_probability_zero / test_explain_probability_one</code>	Probabilidades extremas funcionam
<code>test_build_prompt_with_negative_diagnosis</code>	Prompt inclui "NEGATIVO para SOP" e "15.0%"

test_llm_factory.py (11 testes)

`TestRequireApiKey` — Função `_require_api_key`:

- Chave válida (inclusive com espaços nas bordas) → passa
- String vazia → `LLMConfigurationError` com "credencial"
- String só com espaços → `LLMConfigurationError`

`TestCreateLLMProvider` — Factory `create_llm_provider` para os 4 providers:

Teste	Provider	Mock
test_factory_creates_openai_provider	OpenAI	patch("openai_provider.OpenAI")
test_factory_creates_anthropic_provider	Anthropic	patch("anthropic_provider.anthropic.Anthropic")
test_factory_creates_gemini_provider	Gemini	patch("gemini_provider.openai.Client")
test_factory_creates_groq_provider	Groq	patch("groq_provider.OpenAI")

Cada provider também tem teste de **key missing** → `LLMConfigurationError`.

Provider desconhecido → `ValueError` com "não suportado".

test_llm_providers.py (16 testes)

Três providers (OpenAI, Anthropic, Gemini) × 4 testes cada:

Teste	O que valida
test_provider_name	Nome no formato "provider/modelo"
test_generate_returns_llm_response	Retorna <code>LLMResponse</code> com text + tokens_used
test_generate_passes_correct_params	Parâmetros (model, temperature, max_tokens, messages) são passados corretamente à SDK
test_is_subclass_of_llm_provider	A classe herda de <code>LLMProvider</code>

test_groq_provider.py (4 testes)

Mesmo padrão dos providers acima, específico para Groq.

test_timing_middleware.py (4 testes)

Testa o `TimingMiddleware` do FastAPI:

Teste	O que valida
<code>test_middleware_adds_process_time_header</code>	Resposta inclui <code>X-Process-Time</code>
<code>test_process_time_is_positive_float</code>	Valor é float ≥ 0
<code>test_process_time_increases_with_slower_endpoint</code>	Endpoint <code>/slow</code> (sleep 0.05s) tem tempo maior que <code>/fast</code>
<code>test_middleware_does_not_break_response</code>	Status 200 e body são preservados

Testes de API

Testam os endpoints HTTP usando `TestClient` + `override_deps` para mockar os serviços. Nenhuma chamada real a modelo ou LLM é feita.

`test_predict_api.py` (13 testes)

Classe	Testes	O que validam
<code>TestHealthEndpoint</code>	2	GET <code>/health</code> → 200 + <code>{"status": "ok", "version": "1.0.0"}</code>
<code>TestPredictValidation</code>	6	POST <code>/predict/</code> → 422 para: JSON inválido, campos faltando, <code>follicle_no_r</code> negativo, <code>follicle_no_l</code> negativo, <code>cycle</code> inválido (3), <code>age</code> negativo
<code>TestPredictEndpoint</code>	5	POST <code>/predict/</code> → 200 com mock (estrutura, <code>diagnosis</code> , <code>confidence</code> , <code>top_features</code>) e 503 quando modelo não encontrado
<code>TestExplainValidation</code>	1	POST <code>/explain/</code> → 422 para feature negativa

`test_explain_api.py` (14 testes)

Teste	O que valida
test_explain_com_mock_retorna_200	POST → 200 com explanation, risk_factors, insights
test_explain_com_mock_retorna_texto	Campos com tipos corretos (string, list)
test_explain_com_diagnosis_negativo	diagnosis=0, probability=0.15 → 200
test_explain_sem_features_retorna_422	Campo features ausente → 422
test_explain_sem_diagnosis_retorna_422	Campo diagnosis ausente → 422
test_explain_campos_extra_ignorados	Campos extras no JSON são ignorados
test_explain_menos_de_10_features_retorna_400	< 10 features → 400
test_explain_features_desconhecidas_retorna_400	Nome de feature inválido → 400
test_explain_502_quando_llm_falha	Provider com RuntimeError → 502
test_explain_200_quando_llm_retorna_json_com_markdown	Markdown json ... no retorno → parse OK, 200
test_explain_probabilidade_zero / test_explain_probabilidade_um	Probabilidades 0.0 e 1.0 → 200
test_explain_todas_as_20_features	20 features conhecidas → 200
test_explain_exatamente_10_features	Exatamente 10 features → 200

test_analysis_api.py (10 testes)

Testa o endpoint combinado POST /analysis/ que executa predict + explain:

Teste	O que valida
test_analysis_com_mock_retorna_200	POST → 200
test_analysis_resposta_positivo	diagnosis é "positivo" ou "negativo" (string)

Teste	O que valida
<code>test_analysis_inclui_probabilidade</code>	probability em [0.0, 1.0]
<code>test_analysis_inclui_confidence</code>	confidence é "Alta"/"Média"/"Baixa"
<code>test_analysis_inclui_explanation</code>	explanation é string não vazia, risk_factors e insights são listas
<code>test_analysis_inclui_top_features</code>	top_contributing_features é lista
<code>test_analysis_campos_invalidos_retorna_422</code>	JSON inválido → 422
<code>test_analysis_campos_faltando_retorna_422</code>	Campos obrigatórios ausentes → 422
<code>test_analysis_503_quando_modelo_nao_carregado</code>	Modelo não encontrado → 503
<code>test_analysis_502_quando_explain_falha</code>	LLMRequestError no explain → 502

Testes de Integração

`test_full_pipeline.py` (3 testes)

Usam `override_deps` e testam o fluxo completo da API:

Teste	O que valida
<code>test_health_entao_predict_entao_explain</code>	Encadeia health check → predict → explain, todos 200
<code>test_analysis_completo</code>	<code>/analysis/</code> retorna diagnosis, probability, explanation, top_features
<code>test_todos_endpoints_respondem</code>	Todos os 4 endpoints (health, predict, explain, analysis) retornam 200

Padrões e Convenções

Mock de serviços externos

Toda requisição externa é mockada via **autouse** no conftest:

- AWS Secrets Manager → `get_secret_or_env` retorna `"test_api_key"`
- PostHog → cliente é `None`, chamadas de captura são no-ops

Dependency Override (testes de API)

```
app.dependency_overrides[get_predictor] = lambda: mock_predictor
```

O fixture `override_deps` no conftest já sobrescreve as 4 dependências principais. Testes individuais podem adicionar overrides adicionais em `try/finally` para garantir limpeza.

Testes assíncronos

Usam `@pytest.mark.asyncio` + `AsyncMock`. A engine `asyncio_mode = auto` detecta automaticamente:

```
@pytest.mark.asyncio
async def test_explain_parses_json_response(self):
    result, tokens = await self.service.explain(...)
```

Fakes inline

Classes definidas dentro do próprio arquivo de teste para cenários específicos:

```
class _FakeProvider:
    provider_name = "fake/test"
    def generate(self, system_prompt, user_prompt):
        return '{"explanation": "...", ...}'

class _BrokenProvider:
    provider_name = "broken/test"
    def generate(self, system_prompt, user_prompt):
```

```
raise RuntimeError("API timeout")
```

Fixtures locais vs compartilhadas

- **compartilhadas** (conftest): mocks de serviço, app/client HTTP, dados sintéticos
 - **locais** (no arquivo): `trained_model` (`test_predictor.py`), `dummy_model` (`test_model_registry.py`)
-

Pipeline de CI/CD

O pipeline do GitHub Actions (`.github/workflows/pipeline.yml`) executa nesta ordem:

```
lint → test → build-image → [push-to-ecr] → [deploy-to-ecs]
```

Job `lint`

```
ruff check .  
ruff format --check .
```

Job `test` (dependente de lint)

```
pip install -e ".[dev]"  
pytest --cov=app --cov-report=term-missing
```

- Cobertura é exportada como artifact (`.coverage`)
- Falha em lint ou teste **bloqueia** build e deploy

Tolerância a warnings

Os warnings abaixo são conhecidos e não afetam a execução:

- `StarletteDeprecationWarning`: Using `httpx` with `starlette.testclient` — migração para `httpx2` pendente
- `on_event` is deprecated, use `lifespan` event handlers — migration de `startup/shutdown` para `lifespan`
- `LoggingHandler` is deprecated — migration para `opentelemetry-instrumentation-logging`

Cobertura

Áreas cobertas

Módulo	Cobertura
<code>domain/models.py</code>	Completo (PCOSPrediction, Explanation, FeatureContribution)
<code>domain/enums.py</code>	Completo (DiagnosisStatus, ModelType)
<code>domain/exceptions.py</code>	Completo (3 exceções)
<code>domain/features.py</code>	Validators + <code>validate_explain_features</code> (via API tests)
<code>services/predictor.py</code>	Predição, validação, confidence label, contributions
<code>services/llm_explainer.py</code>	Prompt, parse do JSON, erro do provider
<code>infrastructure/model_registry.py</code>	Carregamento, cache, arquivo inexistente
<code>infrastructure/llm/</code> (4 providers)	Nome, generate, parâmetros, herança
<code>infrastructure/llm/factory.py</code>	Criação, validação de chave, erro
<code>api/v1/predict/</code>	Validação de input, resposta, 503
<code>api/v1/explain/</code>	Validação, LLM error, markdown JSON
<code>api/v1/analysis/</code>	Combinação predict+explain, erros
<code>monitoring/middleware.py</code>	Timing header, performance
<code>core/exceptions.py</code>	<code>AppException</code> + handler
<code>core/config.py</code>	Defaults + env vars

Módulo	Cobertura
core/logger.py	Inicialização com níveis e PostHog

Áreas com cobertura parcial ou pendente

Módulo	Observação
monitoring/posthog.py	Eventos são mockados, funções de captura não têm teste direto
monitoring/metrics.py	setup_metrics() definida mas nunca chamada em main.py — sem teste
infrastructure/secrets_manager.py	Mockado via autouse, sem teste unitário direto
core/dependencies.py	Testado indiretamente via override_deps

Como Executar

```
# Todos os testes
pytest

# Com cobertura
pytest --cov=app --cov-report=term-missing

# Apenas uma camada
pytest tests/unit/
pytest tests/api/
pytest tests/integration/

# Apenas um arquivo
pytest tests/unit/test_predictor.py -v

# Apenas uma classe
pytest tests/unit/test_predictor.py::TestPredictorService -v

# Apenas um teste
pytest
tests/unit/test_predictor.py::TestPredictorService::test_predict_returns_pcos_prediction -v

# Lint + formatação
ruff check .
ruff format --check .
```